

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 3

Problemas NP-Complejos para la defensa de la Tribu del Agua

16 de noviembre de 2025

Agustin Ferronato
111991

Federico Parsons
111250

Axel Alvarado
107679

Ignacio Sebastián Oviedo
109821

1. Introducción

En el presente informe se analizará un problema **NP-Completo**, junto a varios algoritmos obtener la solución óptima o, en su defecto, una aproximación de la misma. Para ello, se hará uso de distintas metodologías vistas durante el desarrollo de la cursada: Backtracking, Programación Lineal Entera y aproximaciones greedy.

Problema: Dado un conjunto de combatientes, donde cada uno está definido por un valor x_i que representa su maestría/fortaleza, y un parámetro k correspondiente a la cantidad de subconjuntos, se busca particionar el conjunto en k subconjuntos de manera de minimizar la adición de los cuadrados de las sumas de las fuerzas de cada subconjunto:

$$\sum_{i=1}^k \left(\sum_{x_j \in S_i} x_j \right)^2$$

2. Preliminares

2.1. Cotas

2.1.1. Cota superior

De la siguiente desigualdad básica:

$$(a + b)^2 = a^2 + 2ab + b^2 > a^2 + b^2 \quad \forall a, b > 0$$

se deduce que agrupar elementos incrementa la suma de cuadrados más que dejarlos separados. Por lo tanto, una cota superior trivial consiste en colocar a todos los combatientes en un único subconjunto:

$$\sum_{i=1}^k \left(\sum_{x_j \in S_i} x_j \right)^2 = \left(\sum_{j=1}^n x_j \right)^2$$

dado que los restantes $k - 1$ subconjuntos tendrían suma cero.

2.1.2. Cota inferior (Desigualdad de Cauchy–Schwarz)

Sean $W, \Pi \in \mathbb{R}^k$ dos vectores de la forma

$$\Pi = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}, \quad W = \begin{bmatrix} W_1 \\ W_2 \\ \vdots \\ W_k \end{bmatrix}, \quad W_i = \sum_{x_j \in S_i} x_j$$

Si definimos a (u, v) como el producto escalar entre los vectores $u, v \in \mathbb{R}^k$, donde:

$$(u, v) = \sum_{i=1}^k u_i v_i$$

se cumple entonces (por Cauchy-Schwarz) la siguiente desigualdad:

$$|(u, v)|^2 \leq |(u, u)| |(v, v)|$$

Si reemplazamos a u por W y a v por Π , obtenemos:

$$\begin{aligned} |(W, \Pi)|^2 &\leq |(W, W)| |(\Pi, \Pi)| \\ \left(\sum_{i=1}^h W_i \right)^2 &\leq \left(\sum_{i=1}^h W_i^2 \right) \left(\sum_{i=1}^h \Pi_i^2 \right) \\ \left(\sum_{i=1}^h W_i \right)^2 &\leq k \left(\sum_{i=1}^h W_i^2 \right) \\ \frac{\left(\sum_{i=1}^k \sum_{x_j \in S_i} x_j \right)^2}{k} &\leq \sum_{i=1}^k \left(\sum_{x_j \in S_i} x_j \right)^2 \\ \frac{\left(\sum_{i=1}^n x_i \right)^2}{k} &\leq \sum_{i=1}^k \left(\sum_{x_j \in S_i} x_j \right)^2 \end{aligned}$$

De aquí se deduce que **toda solución** (óptima o no) siempre tendrá que ser mayor o igual a la expresión dada por:

$$\frac{\left(\sum_{i=1}^n x_i \right)^2}{k}$$

Si tenemos en cuenta la relación entre ambas cotas, llegamos a la siguiente conclusión:

$$\begin{aligned} \text{OPT} &\geq \frac{\left(\sum_{j=1}^n x_j \right)^2}{k}, \quad \text{SOL} \leq \left(\sum_{j=1}^n x_j \right)^2 \\ \frac{\text{SOL}}{\text{OPT}} &\leq \frac{\left(\sum_{j=1}^n x_j \right)^2}{\frac{\left(\sum_{j=1}^n x_j \right)^2}{k}} = k \end{aligned}$$

Es decir, cualquier solución es, en el peor caso, una k -aproximación garantizada, lo cual implica que el problema no admite una aproximación peor que k (no es una ∞ -aproximación).

3. Demostración NP-Compleitud

Para demostrar la NP-Compleitud del problema dado, debemos realizar dos pasos:

1. Demostrar que el problema se encuentra en NP
2. Plantear la reducción de un problema NP-Completo a este.

Denotaremos a partir de aquí a nuestro problema como PTA (problema de la tribu del agua). En primer lugar, demostraremos que:

$$\text{PTA} \in \text{NP}$$

Para ello basta con construir un validador polinomial que dado un conjunto de maestros, un número k , un número B y una lista de k conjuntos de maestros $S_1, S_2, \dots, S_k$, nos permitan determinar si:

$$\sum_{i=1}^k \left(\sum_{x_j \in S_i} x_j \right)^2 \leq B$$

El validador en cuestión es el siguiente:

```
1 def polynomial_verifier(masters, groups, B: int, k: int) -> bool:
2     if len(groups) != k:
3         return False
4
5     # Verificar que los maestros aparezcan una vez por grupo
6     masters_apparences = set()
7     for group in groups:
8         for master in group:
9             if master not in masters_apparences:
10                 masters_apparences.add(master)
11             else:
12                 return False
13
14     if len(masters_apparences) != len(masters):
15         return False
16
17     # Verificar que todos los maestros esten en el conjunto original
18     for group in groups:
19         for master in group:
20             if master not in masters:
21                 return False
22
23     # Verificar si la suma de los cuadrados no excede B
24     total = 0
25     for group in groups:
26         group_sum = 0
27         for master in group:
28             group_sum += master[VALUE_POSITION]
29         total += group_sum ** 2
30     if total > B:
31         return False
32
33     return True
```

La complejidad del validador es $O(n)$, siendo n la cantidad total de maestros (siempre y cuando **masters** sea un set). Notar que, si bien por cada grupo recorro todo el conjunto de maestros de dicho grupo, como cada grupo es un subconjunto y se cumple que: $S = S_1 \cup S_2 \cup \dots \cup S_k$ y $S_1 \cap S_2 \cap \dots \cap S_k = \emptyset$, la iteración por cada uno de ellos me lleva a recorrer $|S| = n$ maestros. De todas formas, si esto no ocurre, el validador lo verifica. Como la iteración se corta cuando se encuentra mas de una aparición de un maestro en dos S_i , en el peor de los casos itero sobre los primeros n maestros y, en el $n + 1$, se corta el algoritmo.

El próximo paso es demostrar la reducción de un problema NP-Completo a PTA. Para ello utilizaremos el problema de 2-partition, que se enuncia como: Dado un conjunto S , determinar si existen dos subconjuntos S_1 y S_2 incluidos en S tal que $S_1 \cup S_2 = S$, $S_1 \cap S_2 = \emptyset$ y que se cumpla que:

$$\sum_{x_i \in S_1} x_i = \sum_{x_j \in S_2} x_j$$

Recordemos que el sentido de la reducción es el siguiente:

$$2\text{-partition} \leq_p \text{PTA}$$

La idea es transformar una instancia del problema de 2-partition a una de PTA. La primera recibe unicamente conjunto S . Determinar si existen dentro de S dos subconjuntos de igual suma es lo mismo que plantear si existe un subconjunto $A \subseteq S$ que cumpla:

$$\sum_{x_i \in A} x_i = \frac{\alpha}{2}, \quad \alpha = \sum_{x_j \in S} x_j$$

Donde (valga la redundancia) el subconjunto $S \setminus A$ cumple:

$$\sum_{x_r \in S \setminus A} x_r = \frac{\alpha}{2}$$

Claramente, el valor de k que espera la instancia de PTA va a ser $k = 2$. A su vez, el valor de B va a ser igual a $\frac{\alpha^2}{2}$ y el conjunto S será el mismo.

Se cumple entonces que: Existen dos subconjuntos de igual suma S_1 y S_2 incluidos en S tal que $S_1 \cup S_2 = S$, $S_1 \cap S_2 = \emptyset \iff$ existen exactamente $k = 2$ subconjuntos en S que cumplan:

$$\sum_{i=1}^{k=2} \left(\sum_{x_j \in S_i} x_j \right)^2 \leq B = \frac{\alpha^2}{2}$$

3.1. Demostración de la ida

La existencia de dos subconjuntos S_1 y S_2 incluidos en S de misma suma que cumplan con lo requerido implica que:

$$\sum_{x_i \in S_1} x_i = \sum_{x_j \in S_2} x_j = \frac{\alpha}{2}$$

Notar que dicha agrupación de ambos subconjuntos cumple que:

$$\begin{aligned} \sum_{i=1}^{k=2} \left(\sum_{x_j \in S_i} x_j \right)^2 &= \left(\sum_{x_r \in S_1} x_r \right)^2 + \left(\sum_{x_m \in S_2} x_m \right)^2 \\ &= \left(\frac{\alpha}{2} \right)^2 + \left(\frac{\alpha}{2} \right)^2 = \frac{\alpha^2}{2} \leq B = \frac{\alpha^2}{2} \end{aligned}$$

Se concluye entonces, de forma practicamente trivial, que la existencia de dos subconjuntos de igual suma implica que existen dos subconjuntos tales que la suma de sus cuadrados es menor o igual a $B = \frac{\alpha^2}{2}$.

3.2. Demostración de la vuelta

Notar que si existen dos subconjuntos incluidos en S que cumplen:

$$\sum_{i=1}^{k=2} \left(\sum_{x_j \in S_i} x_j \right)^2 \leq B = \frac{\alpha^2}{2}$$

el valor de B alcanzado es el minimo posible (demostrado previamente por Cauchy-Schwarz). Por ende, podemos afirmar que es válida la siguiente igualdad:

$$\sum_{i=1}^{k=2} \left(\sum_{x_j \in S_i} x_j \right)^2 = \frac{\alpha^2}{2}$$

$$\left(\sum_{x_r \in S_1} x_r\right)^2 + \left(\sum_{x_m \in S_2} x_m\right)^2 = \frac{\alpha^2}{2}$$

Para simplificar la notación, denotamos por $A = (\sum_{x_r \in S_1} x_r)^2$ y $X = (\sum_{x_m \in S_2} x_m)^2$. Desarrollando la anterior ecuación, llegamos a lo siguiente:

$$A^2 + 2AX + X^2 = \frac{\alpha^2}{2} + 2AX$$

$$(A + X)^2 = \frac{\alpha^2}{2} + 2AX$$

$$\alpha^2 = \frac{\alpha^2}{2} + 2AX$$

$$\frac{\alpha^2}{2} = 2AX$$

$$\frac{\alpha^2}{4} = AX$$

$$\frac{\alpha^2}{4} = A(\alpha - A)$$

$$\frac{\alpha^2}{4} = A\alpha - A^2$$

$$A^2 - A\alpha + \frac{\alpha^2}{4} = 0$$

$$\left(A - \frac{\alpha}{2}\right)^2 = 0$$

$$A = \frac{\alpha}{2}, \quad (A \geq 0)$$

Como $X = (\alpha - A)$, se concluye también que:

$$X = \frac{\alpha}{2}$$

Demostramos de esta forma que si contamos con dos subconjuntos dentro de S cuya suma cuadrática sea menor o igual a $\frac{\alpha^2}{2}$, necesariamente existen dos subconjuntos cuya suma es exactamente la misma.

Entonces, podemos concluir que la reducción:

$$\text{2-partition} \leq_p \text{PTA}$$

es válida. Como 2-partition es un problema NP-Completo (demostrado a continuación), entonces **el problema de la tribu del agua también es NP-Completo.**

4. 2-partition es NP-Completo

Para demostrar que el problema de 2-partition es NP-Completo, utilizaremos un problema visto durante el desarrollo de la cursada que sí lo es: Subset-Sum. Queremos probar si podemos reducir polinomialmente una instancia de Subset-Sum a una de 2-partition, es decir:

$$\text{Subset-Sum} \leq_p \text{2-partition}$$

En principio, hay que demostrar que:

$$\text{2-Partition} \in \text{NP}$$

Para ello, construiremos un validador polinomial que dados S , S_1 y S_2 determine si:

- $S_1 \cap S_2 = \emptyset$
- $S_1 \cup S_2 = S$
- $\sum_{x_i \in S_1} x_i = \sum_{x_j \in S_2} x_j$

```
1 def polynomial_verifier_2_partition(S, S_1, S_2):
2     if len(S_1) + len(S_2) != len(S):
3         return False
4
5     # Verifico si todo elemento de S_1 no esta en S_2 (y viceversa)
6     # y si pertenecen todos a S
7     for x in S_1:
8         if x in S_2 or x not in S:
9             return False
10
11    # Verifico si los elementos de S_2 pertenecen a S
12    for x in S_2:
13        if x not in S:
14            return False
15
16    if S_2.union(S_1) != S:
17        return False
18
19    return sum(S_1) == sum(S_2)
```

La complejidad temporal es $O(n)$, siendo n la cantidad de elementos del conjunto S . Esto ocurre ya que tanto revisar la intersección nula de ambos conjuntos y unir a S_2 con S_1 para luego comparar la union con S , implica recorrer cada uno de los elementos de cada uno de los subconjuntos. Como revisar si un elemento está o no en un conjunto es $O(1)$, no se agrega más complejidad internamente en las iteraciones.

Recordemos el problema de decisión de Subset-Sum: Dado un conjunto S y un valor W , determinar si existe $S' \subseteq S$ que cumpla:

$$\sum_{x_i \in S'} x_i = W$$

Para reducir la instancia original de Subset-sum que recibe un conjunto S y un valor W a una de 2-Partition, creamos un nuevo conjunto A con los mismos elementos de S y le agregamos otros dos elementos: $2\sigma - W$ y $\sigma + W$. Es decir, si $S = \{x_1, x_2, \dots, x_n\}$, entonces:

$$A = \{x_1, x_2, \dots, x_n, 2\sigma - W, \sigma + W\}, \quad \sigma = \sum_{x_i \in S} x_i$$

Ese conjunto es enviado como parametro a la "caja" que resuelve el problema de 2-Partition.

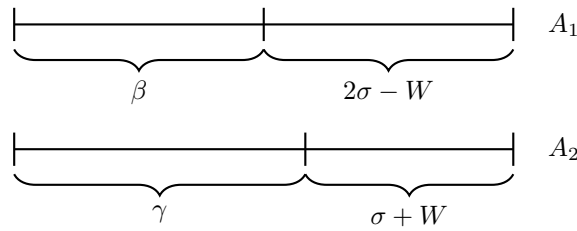
Tenemos que probar que: Existe un subconjunto $S' \subseteq S$ cuya suma es $W \iff$ El conjunto A puede ser particionado en dos subconjuntos A_1 y A_2 cuya suma sea igual, $A_1 \cup A_2 = S$ y $A_1 \cap A_2 = \emptyset$

4.1. Demostración (en ambas direcciones)

Notar que, en la partición de A , los elementos $a = 2\sigma - W$ y $b = \sigma + W$ tienen que estar necesariamente en conjuntos distintos, ya que $a + b = 3\sigma$. Dicha suma nunca va a poder ser alcanzada por el resto de elementos de A , ya que:

$$\sum_{x_i \in A \setminus \{a, b\}} x_i = \sum_{x_j \in S} x_j = \sigma$$

Por ende, nuestra partición va a tener la siguiente forma:



Nos interesa analizar que ocurre si ambas particiones tienen la misma suma:

$$\begin{aligned} \sum_{x_i \in A_1} x_i &= \sum_{x_j \in A_2} x_j \\ \beta + 2\sigma - W &= \gamma + \sigma + W \end{aligned}$$

Recordemos que $\beta + \gamma = \sigma$. Reemplazando a γ por $\sigma - \beta$ y desarrollando la expresión, obtenemos:

$$\begin{aligned} \beta + 2\sigma - W &= \sigma - \beta + \sigma + W \\ 2\beta &= 2W \\ \beta &= W, \quad \gamma = \sigma - \beta \end{aligned}$$

Como β fue construido necesariamente a partir de valores pertenecientes a S , ya que a y b fueron considerados aparte, quiere decir que encontramos un subconjunto dentro de S cuya suma es exactamente igual a W . Es decir, voy a poder particionar a A en dos subconjuntos A_1 y A_2 como los descritos anteriormente $\iff$ existe dentro de S un subconjunto S' cuya suma dé exactamente W . De no ser así, no habría forma de "equilibrar" A_1 y A_2 para que su suma de igual. Por lo tanto, **se deduce que la reducción es correcta y que, en efecto, 2-Partition es NP-Completo**

5. Algoritmo de Backtracking planteado

El problema de minimizar

$$\sum_{i=1}^k \left(\sum_{x_j \in S_i} x_j \right)^2$$

es un problema combinatorial que consiste en distribuir n maestros en k grupos. Puede resolverse mediante un algoritmo de *backtracking* con complejidad $O(k^n)$. A continuación se presenta la implementación de dicho algoritmo, que incorpora diversas podas destinadas a reducir significativamente el tiempo de ejecución. Si bien la complejidad teórica sigue siendo exponencial, la diferencia entre tardar segundos o minutos para un caso con $n \geq 20$ elementos es considerable.

5.1. Optimizaciones

En esta sección se detallan las podas y optimizaciones implementadas.

5.1.1. Poda de Simetrías

Dado que $(a + b)^2 + a^2 = a^2 + (a + b)^2$, si dos o más grupos tienen la misma suma a y se intenta asignar un maestro con valor b , no tiene sentido explorar todas las combinaciones simétricas equivalentes. Esta poda evita explorar permutaciones redundantes que no generan soluciones distintas.

5.1.2. Optimización por Ordenamiento

Se ordena previamente el arreglo de maestros de forma descendente según su peso (o valor). De esta manera, las decisiones con mayor impacto se toman primero, lo que mejora la eficiencia de las podas posteriores.

5.1.3. Optimización mediante una Solución Inicial Aproximada

Si bien el valor inicial de `best` podría ser cualquier solución válida (por ejemplo, colocar a todos los maestros en un único grupo, que representa el peor escenario), se utiliza un algoritmo de aproximación (ver Sección 7) para generar una solución inicial relativamente buena, reduciendo así el espacio de búsqueda.

5.1.4. Poda por Rama Mínima

Además de pasar el estado actual, se transmite también la suma de los valores que aún quedan por asignar. Esto permite calcular un límite inferior estimado asumiendo que los elementos restantes (incluso bajo la distribución más equilibrada posible) no podrían generar un resultado mejor que el actual. Si este límite inferior ya es mayor o igual al valor actual de `best`, la rama se poda inmediatamente, pues no puede mejorar la mejor solución conocida.

5.2. Implementacion

```
1 def bt(masters: list[Master], k: int, index: int, current, best):
2     sum_of_each_set, current_list, current_sum, remains = current
3     best_sum, best_list = best
4
5     if len(masters) == index:
6         if current_sum < best_sum:
7             return current_sum, copy.deepcopy(current_list)
8
9     if current_sum >= best_sum:
10        return best_sum, best_list
11
12    if lower_bound(sum_of_each_set, remains) >= best_sum:
13        return best_sum, best_list
14
15    master = masters[index]
16    master_value = master[VALUE_POSITION]
17    visited_values = set()
18
19    for i in range(k):
20        if sum_of_each_set[i] in visited_values:
21            continue
22
23        visited_values.add(sum_of_each_set[i])
24
25        current_list[i].append(master)
26
27        remains -= master_value
28        current_sum -= sum_of_each_set[i] ** 2
29        sum_of_each_set[i] += master_value
30        current_sum += sum_of_each_set[i] ** 2
31
32        current = (sum_of_each_set, current_list, current_sum, remains)
33        best = bt(masters, k, index + 1, current, best)
34
35        remains += master_value
36        current_sum -= sum_of_each_set[i] ** 2
37        sum_of_each_set[i] -= master_value
38        current_sum += sum_of_each_set[i] ** 2
39
40        current_list[i].pop()
41
42    return best
43
44
45 def lower_bound(sum_of_each_set: list[int], remains: int) -> int:
46     sums = sorted(sum_of_each_set)
47     k = len(sums)
48
49     for i in range(k - 1):
50         if remains == 0:
51             break
52         diff = sums[i + 1] - sums[i]
53         block = i + 1
54         need = diff * block
55         if remains >= need:
56             for j in range(block):
57                 sums[j] += diff
58                 remains -= need
59         else:
60             full = remains // block
61             rest = remains % block
62             for j in range(block):
63                 sums[j] += full
64             for j in range(rest):
65                 sums[j] += 1
66             remains = 0
67     if remains > 0:
68         full = remains // k
```

```
69     rest = remains % k
70     for i in range(k):
71         sums[i] += full
72     for i in range(rest):
73         sums[i] += 1
74
75     return sum(s**2 for s in sums)
76
```

5.3. Mediciones de tiempos

Con la finalidad de comprobar empíricamente el comportamiento exponencial del algoritmo, se realizaron diversas ejecuciones para valores de entrada desde $n = 5$ hasta $n = 30$, con $k = \lfloor n/2 \rfloor$. El valor de k seleccionado para cada instancia no es arbitrario, sino que representa un valor crítico para el algoritmo en sí. Si k toma valores cercanos a 1 o a n , tenemos menos posibilidades de elección que puedan afectar al valor final de la suma, por lo que el algoritmo tendrá un menor tiempo de ejecución.

Las mediciones fueron separadas en dos gráficos: el primero toma valores desde $n = 5$ hasta $n = 15$, y el segundo, $n = 15$ hasta $n = 30$. El motivo de la división fue que, al unificar ambos gráficos, no se llegaba a apreciar la diferencia temporal entre las ejecuciones de la primera instancia.

Además, se fijó $n = 30$ como valor máximo de análisis, dado que para $n > 30$ el tiempo de ejecución del algoritmo se volvía prácticamente prohibitivo, tornando inviables nuevas mediciones. Eso nos da una noción clara del comportamiento exponencial del algoritmo: una mínima diferencia en el tamaño de entradas puede causar una diferencia de incluso varias horas en los tiempos de ejecución.

Se muestran en la figuras 1 y 2 las ejecuciones para los valores de entrada mencionados, respectivamente.

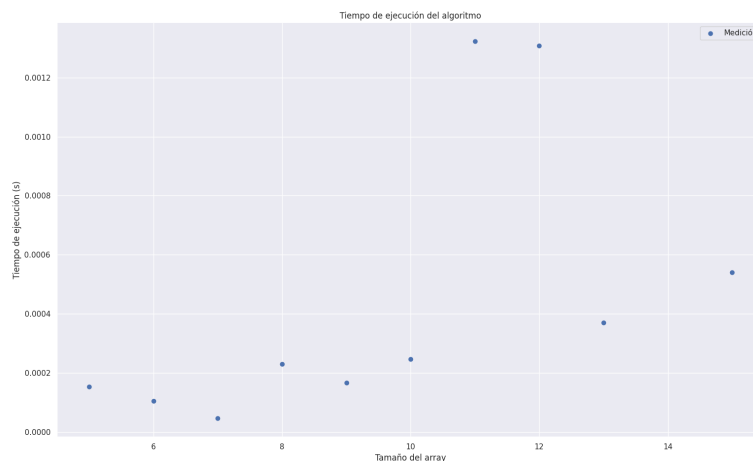


Figura 1: Mediciones realizadas desde $n = 5$ hasta $n = 15$

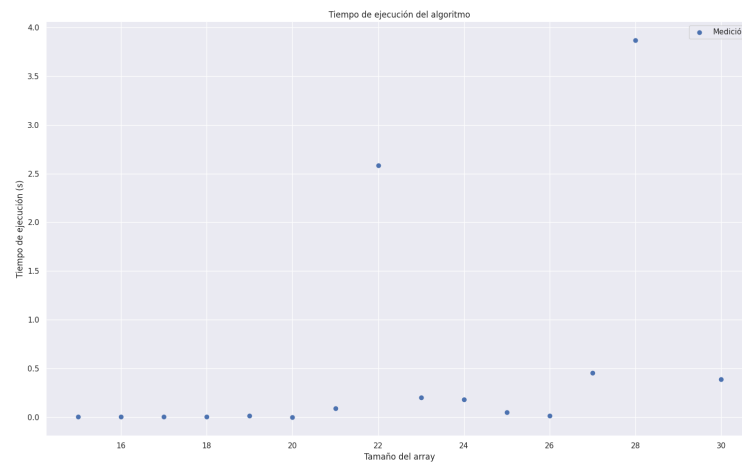


Figura 2: Mediciones realizadas desde $n = 15$ hasta $n = 30$

6. Modelo de programación lineal

6.1. Modelo que obtiene la solución óptima

Sea $M_{i,j} \in \{0, 1\}$ la variable que indica que el maestro i -ésimo se encuentra en el subconjunto $S_j \subseteq S$. Naturalmente, un maestro va a poder pertenecer a un único subconjunto, por lo que debemos agregar la siguiente restricción:

$$\sum_{j=1}^{|S|} M_{i,j} = 1 \quad \forall i = 1, 2, \dots, n$$

Como el problema original busca minimizar la suma de los cuadrados de cada uno de dichos subconjuntos, nos vemos obligados a crear mas variables y restricciones para "linealizar" la función objetivo.

Sea entonces $E_{r,m,k} \in \{0, 1\}$ aquella variable que toma el valor de 1 si los maestros r y m se encuentran en el mismo subconjunto S_k , y 0 en caso contrario.

Notar que hay una clara relación entre $M_{i,j}$ y $E_{r,m,k}$. La variable $E_{r,m,k} = 1 \iff M_{r,k} = 1$ y $M_{m,k} = 1$. Para representar esta relación en nuestro modelo, planteamos la siguiente restricción:

$$2E_{r,m,k} \leq M_{r,k} + M_{m,k} \leq 1 + E_{r,m,k} \\ \forall r < m, \forall k = 1, 2, \dots, |S| \quad r, m = 1, 2, \dots, n$$

Se verifica facilmente (reemplazando los valores) que $E_{r,m,k} = 1$ unicamente cuando $M_{r,k} = 1$ y $M_{m,k} = 1$. Además, es importante que $r < m$, ya que las variables $E_{r,m,k}$ y $E_{m,r,k}$ son equivalentes. Representan exactamente lo mismo para el modelo, y la coexistencia de ambas unicamente agregaría más restricciones. Lo mismo ocurre con variables del estilo $E_{m,m,k}$, por lo que no son consideradas.

Con este sencillo truco, la función objetivo del modelo queda definida por:

$$\min \left\{ \sum_{k=1}^{|S|} \sum_{r=1}^n \sum_{m=1}^n 2E_{r,m,k} x_r x_m + \sum_{r=1}^n x_r^2 \right\}$$

Se podría obviar la última sumatoria, puesto que siempre vamos a tener que sumar todos los cuadrados de los maestros, independientemente de que configuración de subconjuntos haya. Notar que la función objetivo es simplemente el desarrollo de la suma de cuadrados, la cual se adapta a cualquier configuración de subconjuntos.

Además, podríamos agregar una nueva restricción que nos garantice evitar simetrías en cuanto a las sumatorias de los subconjuntos. Es decir, como para el estado parcial del modelo es indistinto si, por ejemplo, $S_j = \{4, 1\}$ y $S_p = \{3\}$ o $S_j = \{3\}$ y $S_p = \{4, 1\}$, evitamos contemplar ambas situaciones. Esto lo logramos de la siguiente manera:

$$\sum_{i=1}^n x_i M_{i,r} \leq \sum_{j=1}^n x_j M_{j,r+1}, \quad r = 1, 2, \dots, |S| - 1$$

En total, el modelo cuenta con $n + |S|n(\frac{n-1}{2}) + |S| - 1$ restricciones. Esto viene dado por las primeras n igualdades correspondientes a las variables $M_{i,j}$, las $|S| - 1$ restricciones que evitan las simetrías en la suma de los subconjuntos y las $|S|n(\frac{n-1}{2})$ restricciones vinculadas a las variables $E_{r,m,k}$. Esto último ocurre ya que para cada subconjunto posible, itero sobre los n maestros originales. A su vez, cada maestro considera unicamente a los de mayor subíndice, por lo que voy a tener variables unicamente del tipo $E_{r,m,k}$ con $r < m$, evitando simetrías y, por ende, duplicar las variables de manera innecesaria.

6.2. Modelo aproximado

Para el modelo aproximado propuesto por la catedra, haremos uso también de la variable $M_{i,j}$ de la sección anterior. Como la función objetivo (presentada mas adelante) no posee sumas cuadráticas per se, no será de necesidad la creación de muchas variables de decisión como el modelo que obtiene la solución óptima. Como un maestro i puede estar en un único conjunto j , planteamos la misma restricción que el modelo anterior:

$$\sum_{j=1}^{|S|} M_{i,j} = 1 \quad \forall i = 1, 2, \dots, n$$

Sea $T_j \in \mathbb{N}$, la variable que representa la suma del subconjunto $S_j \subseteq S$. Se cumple que (valga la redundancia):

$$T_j = \sum_{i=1}^n x_i M_{i,j}, \quad \forall j = 1, 2, \dots, |S|$$

Definimos tambien a $W = \max\{T_j\}$ y a $m = \min\{T_j\}$. En terminos del modelo, esto implica que:

$$W \geq T_j, \quad m \leq T_j, \quad \forall j = 1, 2, \dots, |S|$$

Como la función objetivo implica restar el grupo de mayor suma con el de menor suma, su estructura es la siguiente:

$$\min\{W - m\}$$

Por supuesto que esto no garantiza la solución óptima, pero puede considerarse una buena aproximación a priori ya que busca "balancear" la suma de los subconjuntos: distribuirlos de manera uniforme.

La cantidad de restricciones es $n+3|S|$, notoriamente menor al modelo propuesto anteriormente. Esto se debe a que tenemos n restricciones correspondientes a la sumatoria que garantiza que un maestro pueda estar unicamente en un subconjunto. Además, contamos con $|S|$ igualdades propias de los T_j , mas las $2|S|$ inecuaciones planteadas para encontrar el máximo y el mínimo de dichas T_j .

7. Aproximación Greedy (propuesta del Maestro Pakku)

Sabiendo que nuestro problema es NP-Completo en su versión de decisión, hallar una solución óptima al mismo en tiempo polinomial (en su versión de optimización) es prácticamente imposible (o así se cree). Es por este motivo que se propone un algoritmo de aproximación que obtiene una solución con una cota aceptable. El algoritmo en cuestión es el siguiente:

- Ordenar los maestros por su fortaleza de mayor a menor
- Asignar a los maestros (siguiendo dicho orden) en el subgrupo de mínima suma cuadrática

Notar que el algoritmo es el mismo que el problema del balanceo de carga visto en clase (**Longest Process Time Scheduling**). Esto nos será de utilidad para el cálculo de la cota para el peor caso de la aproximación.

7.1. Implementación y complejidad

La complejidad del algoritmo implementado es:

$$T(n, k) = O(n \log(n))$$

Esto ocurre ya que el algoritmo ordena en primera instancia, y hace uso de un *heap de mínimos* para la búsqueda del subconjunto de suma mínima. Es decir, el desarrollo del cálculo de la complejidad sería:

$$T(n, k) = O(n \log(n) + n \log(k)) = O(n(\log(n) + \log(k))) = O(n \log(kn))$$

El factor $n \log(k)$ es agregado ya que, en cada iteración, voy a tener que consultar el mínimo del heap y reinsertar dicho valor actualizado, lo cual cuesta $O(\log(k))$. Sabiendo que $k = cn$, siendo c una constante que cumple $\frac{1}{n} \leq c \leq 1$, reemplazamos:

$$T(n, k) = O(n \log(cn)) = O(n \log(cn^2)) = O(2n \log(cn)) = O(n \log(n))$$

La implementación del algoritmo es la siguiente:

```
1 def greedy(masters: list[Master], k: int):
2     sums: list[int] = [0] * k
3     heap = [(0, i) for i in range(k)]
4     group: list = [[] for _ in range(k)]
5     heapq.heapify(heap)
6     for master in masters:
7         current_sum, i = heapq.heappop(heap)
8         group[i].append(master)
9         new_sum = current_sum + master[VALUE_POSITION]
10        heapq.heappush(heap, (new_sum, i))
11        sums[i] = new_sum
12
13    return group
14
15 def pakku(masters: list[Master], k: int):
16     masters = sorted(masters, key=lambda master: master[VALUE_POSITION], reverse=True)
17     return greedy(masters, k)
18
```

Se realizaron mediciones para corroborar empíricamente la complejidad. Los sets de datos utilizados fueron de un tamaño desde $n = 1000$ hasta $n = 100000$, con $k = \lfloor n/2 \rfloor$, para un total de 45 muestras. Los valores de fortaleza de los maestros son aleatorios, en un rango que va desde 50 hasta 1750. Se observa en la figura 3 el ajuste por mínimos cuadrados de los tiempos de ejecución para cada entrada con la función $T(n) = n \log(n)$. Para dicho ajuste, se obtuvo un error cuadrático

medio $\epsilon \approx 0,0008324$. Además, se comparo la medición con otras funciones que no corresponden a la complejidad teórica indicada: $T(n) = n$ y $T(n) = n^2$. Se observa en la figura 4 la comparativa entre los errores para las tres funciones.

El error cuadrático obtenido para las ultimas dos fue de $\epsilon' \approx 0,0012007$ y $\epsilon'' \approx 0,00221312$, respectivamente.

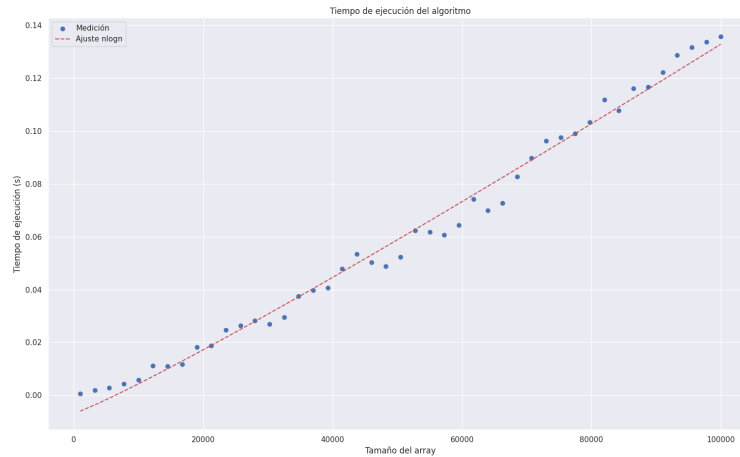


Figura 3: Ajuste del algoritmo para la función $T(n) = n \log(n)$

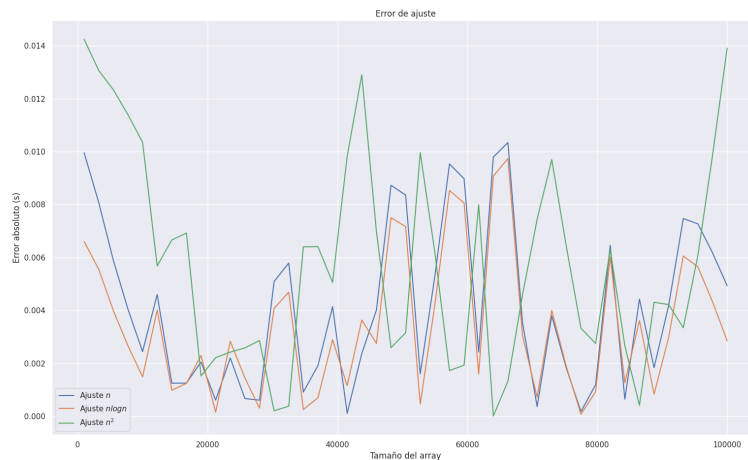


Figura 4: Error de ajuste para cada una de las funciones

7.2. Análisis del factor de aproximación

Como mencionamos previamente, el algoritmo es el mismo que **Longest Process Time Scheduling** (LPT), por lo que realizaremos un análisis similar. Notar que si bien la función objetivo de ambos problemas es distinta (el nuestro busca minimizar la adición de la suma de los cuadrados de cada uno de los subconjuntos y, utilizando LPT, la máquina con mayor cantidad de trabajo), el propósito e interés del algoritmo tiene la misma aplicación para ambos problemas, ya que buscamos balancear la carga entre cada uno de los subconjuntos.

Sea C_m la carga obtenida a la hora de planificar los trabajos con LPT y sea C_m^* la planificación óptima. Se puede probar que:

$$C_m \leq \left(\frac{4}{3} - \frac{1}{3m}\right) C_m^* \quad [\text{https://elementsofscheduling.nl, chap. 8}]$$

Siendo m la cantidad de máquinas (k en nuestro caso). Entonces, si logramos encontrar una configuración tal que alcance esa cota máxima, habremos encontrado el peor caso de asignación, y podríamos hallar una cota para nuestro problema original.

Consideremos $|S| = 2k + 1$ elementos, y la siguiente configuración para ellos:

$$S = \{2k - 1, 2k - 1, 2k - 2, 2k - 2, \dots, k, k, k\}$$

Para dicho caso, el resultado óptimo de asignación es:

$$S_1^* = \{k, k, k\}, \quad S_2^* = \{2k - 1, k + 1\}, \quad S_3^* = \{2k - 1, k + 1\}, \quad \dots, \quad S_k^* = \left\{\frac{3k}{2}, \frac{3k}{2}\right\}.$$

Y el obtenido por nuestro algoritmo:

$$S_1 = \{2k-1, k, k\}, \quad S_2 = \{2k-1, k\}, \quad S_3 = \{2k-2, k+1\}, \quad \dots, \quad S_{k-1} = \left\{\frac{3k}{2}, \frac{3k}{2} - 1\right\}, \quad S_k = \left\{\frac{3k}{2}, \frac{3k}{2} - 1\right\}.$$

Donde

$$C_m = 4k - 1, \quad C_m^* = 3k$$

Por lo tanto,

$$\frac{C_m}{C_m^*} = \frac{4k - 1}{3k} = \frac{4}{3} - \frac{1}{3k}$$

Podemos concluir que la asignación generada por nuestro algoritmo corresponde al peor caso posible, es decir, aquella que produce el mayor desequilibrio entre los subconjuntos.

Sea T^* la sumatoria del conjunto óptimo, y T , la del algoritmo greedy. Si desarrollamos ambas expresiones, obtenemos:

$$T^* = \sum_{i=1}^k (3k)^2 = k \cdot (3k)^2 = 9k^3.$$

$$T = (4k - 1)^2 + (k - 1)(3k - 1)^2 = 9k^3 + k^2 - k.$$

$$r_p(k) = \frac{T}{T^*} = \frac{9k^3 + k^2 - k}{9k^3} = 1 + \frac{1}{9k} - \frac{1}{9k^2}, \quad 1 \leq k \leq |S|$$

Demostramos de tal manera que el algoritmo propuesto por el maestro Pakku es una $r_p(k)$ -aproximación.

Para corroborar empíricamente la cota para volúmenes de datos ya inmanejables por los algoritmos exactos (Backtracking y Programación Lineal), generamos distintos conjuntos de datos como los descritos anteriormente en esta sección. Es decir, sets de tamaño $2k + 1$, con k variando entre 2000 y 15000.

Dicha comprobación se visualiza en la tabla 1. Notar que pueden existir discrepancias entre el valor esperado de $r_p(k)$ y el valor obtenido. Esto ocurre simplemente por una diferencia en la precisión en el cálculo de uno y otro valor. Para evidenciar que dicha diferencia es despreciable, el radio se considera igual al esperado en un rango de $r_p(k) \pm \epsilon$, con $\epsilon = 1 \times 10^{-12}$.

k	Execution time (s)	Ratio (SOL/OPT)	Expected ratio	Ratio = Expected
2000	0.0016951561	1.0000555278	1.0000555278	True
2500	0.0020320415	1.0000444267	1.0000444267	True
3000	0.0025415421	1.0000370247	1.0000370247	True
3500	0.0030877590	1.0000317370	1.0000317370	True
4000	0.0033006668	1.0000277708	1.0000277708	True
4500	0.0037434101	1.0000246859	1.0000246859	True
5000	0.0043635368	1.0000222178	1.0000222178	True
5500	0.0046882629	1.0000201983	1.0000201983	True
6000	0.0051546097	1.0000185154	1.0000185154	True
6500	0.0054724216	1.0000170914	1.0000170914	True
7000	0.0064914227	1.0000158707	1.0000158707	True
7500	0.0082166195	1.0000148128	1.0000148128	True
8000	0.0075988770	1.0000138872	1.0000138872	True
8500	0.0076718330	1.0000130704	1.0000130704	True
9000	0.0098721981	1.0000123443	1.0000123443	True
9500	0.0085747242	1.0000116947	1.0000116947	True
10000	0.0092220306	1.0000111100	1.0000111100	True
10500	0.0094795227	1.0000105810	1.0000105810	True
11000	0.0098733902	1.0000101001	1.0000101001	True
11500	0.0120627880	1.0000096610	1.0000096610	True
12000	0.0108025074	1.0000092585	1.0000092585	True
12500	0.0130786896	1.0000088882	1.0000088882	True
13000	0.0116398335	1.0000085464	1.0000085464	True
13500	0.0143084526	1.0000082298	1.0000082298	True
14000	0.0130181313	1.0000079359	1.0000079359	True
14500	0.0151822567	1.0000076623	1.0000076623	True
15000	0.0134172439	1.0000074069	1.0000074069	True

Cuadro 1: Ejecuciones del algoritmo de Pakku para volúmenes de datos grandes

A su vez, se realizó una comparación similar generando conjuntos de datos cuya solución óptima se conoce de antemano. Para lograr esto, generamos los k subconjuntos y asignamos a todos ellos un mismo valor aleatorio. Internamente, "creamos" maestros con una fortaleza también aleatoria, y la distribuimos entre los maestros restantes de dicho subconjunto. Cada grupo posee una cantidad de $\lfloor \frac{n}{k} \rfloor$ maestros. Si el número $\frac{n}{k}$ no es entero, se asignan los restantes $n \bmod k$ al último conjunto.

El motivo de dicho mecanismo para crear conjuntos de datos es el de no caer en una lógica circular a la hora de comparar resultados, y tener una noción real de cuán buena es la aproximación propuesta.

En la tabla 2 se visualiza la misma comparación que la realizada en el cuadro 1, para valores de entrada que varían desde $n = 200$ hasta $n = 300$, con un valor de $k = \lfloor \frac{n}{2} \rfloor$. Si bien los tamaños de los sets son menores, siguen siendo prácticamente inmanejables por los algoritmos exactos.

n	Execution time (s)	Ratio (SOL/OPT)	Max ratio	Ratio $\leq$ Max
201	0.000093	1.00010204	1.00110	True
203	0.000079	1.00002552	1.00108922	True
205	0.000083	1.00000	1.00107864	True
207	0.000073	1.00004372	1.00106828	True
209	0.000076	1.00000082	1.00105810	True
211	0.000075	1.00001037	1.00104812	True
213	0.000073	1.00008163	1.00103833	True
215	0.000079	1.00001472	1.00102872	True
217	0.000076	1.00000680	1.00101928	True
219	0.000082	1.00000238	1.00101002	True
221	0.000081	1.00005505	1.00100092	True
223	0.000086	1.00000	1.00099198	True
225	0.000083	1.00005177	1.00098321	True
227	0.000088	1.00000557	1.00097458	True
229	0.000085	1.00014499	1.00096611	True
231	0.000087	1.00003996	1.00095778	True
233	0.000085	1.00002591	1.00094960	True
235	0.000093	1.00009694	1.00094155	True
237	0.000089	1.00002769	1.00093364	True
239	0.000089	1.00005249	1.00092586	True
241	0.000090	1.00000	1.00091821	True
243	0.000093	1.00001080	1.00091068	True
245	0.000089	1.00000243	1.00090328	True
247	0.000093	1.00000428	1.00089600	True
249	0.000090	1.00002852	1.00088883	True
251	0.000093	1.00008	1.00088178	True
253	0.000093	1.00002405	1.00087484	True
255	0.000095	1.00001362	1.00086800	True
257	0.000097	1.00002555	1.00086127	True
259	0.000099	1.00000022	1.00085465	True
261	0.000101	1.00000206	1.00084813	True
263	0.000103	1.00000496	1.00084170	True
265	0.000111	1.00001247	1.00083537	True
267	0.000107	1.00000	1.00082914	True
269	0.000108	1.00000	1.00082300	True
271	0.000104	1.00001273	1.00081695	True
273	0.000110	1.00001218	1.00081099	True
275	0.000113	1.00001780	1.00080511	True
277	0.000120	1.00001288	1.00079932	True
279	0.000110	1.00000986	1.00079361	True
281	0.000119	1.00000195	1.00078798	True
283	0.000112	1.00001732	1.00078243	True
285	0.000111	1.00001139	1.00077696	True
287	0.000112	1.00000375	1.00077157	True
289	0.000134	1.00000	1.00076625	True
291	0.000121	1.00000589	1.00076100	True
293	0.000136	1.00000053	1.00075582	True
295	0.000113	1.00002015	1.00075072	True
297	0.000109	1.00000629	1.00074568	True
299	0.000104	1.00000493	1.00074071	True

Cuadro 2: Ejecución del algoritmo de Pakku con la nueva estrategia de generación de datos

8. Ejecución y comparación de los algoritmos

Para verificar de manera empírica tanto los tiempos de ejecución como las cotas obtenidas en los modelos aproximados, se realizaron diversos ejemplos de ejecución. Se generaron sets de datos con $n = 2, \dots, 13$ maestros y, para cada valor de n , se realizaron pruebas con $k = 2, 3, \dots, n - 2$. Las fortalezas de cada uno de los maestros toma un valor entero aleatorio entre 50 y 1750.

Las medidas de comparación utilizadas fueron:

- Tiempos de ejecución de cada algoritmo (Figura 3)
- Coeficiente entre la solución óptima para el set de datos en particular y la solución obtenida mediante dicho algoritmo. Es decir, si I es nuestra instancia del problema, $z(I)$ su solución óptima y $A(I)$ la solución obtenida, nos interesa analizar el valor del coeficiente $r(A) = \frac{A(I)}{z(I)}$ (Figura 4)

De los resultados empíricos podemos obtener varias conclusiones. En primer lugar, era esperado que para toda instancia del problema I , nuestros algoritmos de Backtracking y Programación Lineal obtuvieran un coeficiente $r(A) = \frac{A(I)}{z(I)} = 1$. De no ser así, habrían sido mal codificados. Además, se ve una importante diferencia entre los tiempos de ejecución de Backtracking con los de Programación Lineal. Esto se debe principalmente a las podas implementadas, que optimizaron prácticamente al máximo el primer algoritmo mencionado.

Notar que, para un valor de n dado, los tiempos de ejecución para el modelo de programación lineal aumentan a medida que el valor de k se acerca a la media de dicho n , y vuelven a disminuir cuando se acerca a n . Esto es bastante intuitivo: para valores tanto "chicos" como "grandes" de k tengo menos opciones a la hora de distribuir a los maestros en los distintos subconjuntos, por lo que el modelo tendrá que considerar muchísimas menos ramas a la hora de resolverse.

Podemos observar incluso que se cumple la cota descrita en la sección anterior para la aproximación greedy:

$$r(A) \leq 1 + \frac{1}{9k} - \frac{1}{9k^2}, \quad \forall k$$

n	k	Approx. LP	LP	Backtracking	Paku
5	2	0.007507	0.021878	0.00004911	0.00000525
5	3	0.005831	0.072029	0.00006652	0.00000620
6	2	0.006227	0.052489	0.00010896	0.00000572
6	3	0.006121	0.235978	0.00006747	0.00000644
6	4	0.006371	0.329338	0.00008178	0.00000691
7	2	0.006740	0.280890	0.00011349	0.00000644
7	3	0.007596	0.423966	0.00025177	0.00000882
7	4	0.006616	0.516555	0.00008106	0.00000763
7	5	0.006835	0.179300	0.00009179	0.00000811
8	2	0.006604	0.350238	0.00014353	0.00000668
8	3	0.006642	0.533926	0.00022531	0.00000739
8	4	0.006900	0.941762	0.00010061	0.00000787
8	5	0.007020	0.396929	0.00008917	0.00000906
8	6	0.007056	0.354893	0.00007749	0.00000858
9	2	0.006181	0.395532	0.00018406	0.00000691
9	3	0.006559	1.494737	0.00045800	0.00001144
9	4	0.007672	1.083499	0.00011706	0.00000834
9	5	0.008026	1.538092	0.00034380	0.00000954
9	6	0.007319	0.744012	0.00017285	0.00000930
9	7	0.007397	0.560742	0.00010777	0.00000978
10	2	0.007181	0.596096	0.00030065	0.00000739
10	3	0.006778	2.281647	0.00062108	0.00000978
10	4	0.007711	1.257638	0.00019169	0.00000954
10	5	0.007487	1.661477	0.00024009	0.00000930
10	6	0.007415	4.512093	0.00023556	0.00001168
10	7	0.008072	2.007347	0.00017715	0.00001001
10	8	0.007458	0.509446	0.00011706	0.00000954
11	2	0.006198	0.749677	0.00039244	0.00000858
11	3	0.006513	4.063656	0.00087738	0.00001073
11	4	0.007773	5.915874	0.00146294	0.00001812
11	5	0.007955	7.143268	0.00015020	0.00000930
11	6	0.008001	3.031182	0.00044107	0.00001144
11	7	0.007558	3.452801	0.00013709	0.00001049
11	8	0.008563	2.342396	0.00023770	0.00001335
11	9	0.009033	1.599706	0.00014067	0.00001216
12	2	0.006478	1.231854	0.00070524	0.00001025
12	3	0.007015	6.782549	0.00091338	0.00001049
12	4	0.008306	16.184214	0.00419879	0.00006747
12	5	0.007651	40.851563	0.00059581	0.00001431
12	6	0.008758	13.421826	0.00019407	0.00001216
12	7	0.009008	107.705822	0.00033998	0.00001073
12	8	0.007579	2.782765	0.00018382	0.00001049
12	9	0.007463	3.451627	0.00017238	0.00001073
12	10	0.007432	1.819134	0.00013828	0.00001121
13	2	0.006001	1.557647	0.00136328	0.00001454
13	3	0.006289	6.872088	0.00344777	0.00006104
13	4	0.007326	66.793736	0.01327181	0.00002289
13	5	0.007717	98.740969	0.00233316	0.00001955
13	6	0.008485	45.725070	0.00034618	0.00001192
13	7	0.008675	408.034800	0.00124073	0.00001454
13	8	0.007448	285.173259	0.00028062	0.00001311
13	9	0.009377	27.467117	0.00027800	0.00001550
13	10	0.009909	6.715965	0.00032663	0.00001502
13	11	0.009997	1.980494	0.00016141	0.00001144

Cuadro 3: Tiempos de ejecución (en segundos) para distintos (n, k) para cada algoritmo

n	k	Approx. LP	LP	Backtracking	Pakku
5	2	1.999353	1.0	1.0	1.0
5	3	1.691691	1.0	1.0	1.0
6	2	1.999981	1.0	1.0	1.001834
6	3	1.070515	1.0	1.0	1.0
6	4	1.467582	1.0	1.0	1.0
7	2	1.020420	1.0	1.0	1.000012
7	3	1.506071	1.0	1.0	1.005481
7	4	1.940747	1.0	1.0	1.0
7	5	1.805697	1.0	1.0	1.0
8	2	1.290811	1.0	1.0	1.000176
8	3	1.598903	1.0	1.0	1.001457
8	4	2.0366	1.0	1.0	1.0
8	5	2.0428	1.0	1.0	1.0
8	6	2.7381	1.0	1.0	1.0
9	2	1.024583	1.0	1.0	1.003447
9	3	1.550919	1.0	1.0	1.001724
9	4	2.0311	1.0	1.0	1.0
9	5	1.571355	1.0	1.0	1.0
9	6	2.8418	1.0	1.0	1.000137
9	7	2.5173	1.0	1.0	1.0
10	2	1.022331	1.0	1.0	1.000338
10	3	1.209642	1.0	1.0	1.001276
10	4	2.1808	1.0	1.0	1.000288
10	5	2.6666	1.0	1.0	1.0
10	6	1.899339	1.0	1.0	1.0
10	7	2.2826	1.0	1.0	1.0
10	8	1.922014	1.0	1.0	1.0
11	2	1.008622	1.0	1.0	1.000117
11	3	1.504084	1.0	1.0	1.000654
11	4	1.563678	1.0	1.0	1.000870
11	5	2.4552	1.0	1.0	1.0
11	6	2.1680	1.0	1.0	1.000575
11	7	1.779984	1.0	1.0	1.0
11	8	1.800154	1.0	1.0	1.0
11	9	1.984748	1.0	1.0	1.0
12	2	1.000412	1.0	1.0	1.000007
12	3	1.659737	1.0	1.0	1.000425
12	4	1.848424	1.0	1.0	1.003491
12	5	2.0720	1.0	1.0	1.001068
12	6	2.7650	1.0	1.0	1.0
12	7	2.8449	1.0	1.0	1.0
12	8	2.7657	1.0	1.0	1.0
12	9	3.1406	1.0	1.0	1.0
12	10	3.7451	1.0	1.0	1.0
13	2	1.089119	1.0	1.0	1.000719
13	3	1.557119	1.0	1.0	1.000123
13	4	2.1556	1.0	1.0	1.000790
13	5	2.1733	1.0	1.0	1.000844
13	6	2.4700	1.0	1.0	1.0
13	7	3.0297	1.0	1.0	1.0
13	8	2.6464	1.0	1.0	1.0
13	9	2.6928	1.0	1.0	1.0
13	10	3.3910	1.0	1.0	1.0
13	11	2.7309	1.0	1.0	1.0

Cuadro 4: Valor del coeficiente $r(A)$ para cada instancia

9. Conclusiones

Demostramos como el Problema de la Tribu del Agua (PTA) es NP-Completo. Es por ello que obtener la solución óptima (para un conjunto de datos cualquiera) en tiempo polinomial es prácticamente imposible (a menos que $P = NP$). Para llegar a la misma, considerando datos de entrada de tamaño chico, hemos implementado algoritmos de Backtracking y Programación Lineal. La naturaleza combinatoria del problema lleva a que dichos algoritmos sean inviables para tamaños de entrada de $n > 30$, por lo que se realizó el análisis de dos posibles aproximaciones: la propuesta por el maestro Pakku y un modelo de Programación Lineal con una función objetivo distinta a la definida en el modelo que llega a la solución óptima del problema, con la finalidad de tener una cantidad menor de restricciones.

Durante el análisis del algoritmo de Pakku, se llegó a la conclusión de que es una $r_p(k)$ -aproximación, siendo esta una cota **muy buena**, sobre todo para conjuntos de datos que resultan inmanejables por los algoritmos que garantizan la solución exacta del problema. Notar que si $k \rightarrow \infty \Rightarrow r_p(k) \rightarrow 1$. Esto se evidencia en el cuadro 1 (teniendo en cuenta además que los sets utilizados para dicho cuadro son los mas desfavorables posibles para el algoritmo). Incluso, se observa en el cuadro 4 que existen varios escenarios en los que la solución óptima coincide con la obtenida por medio del algoritmo de Pakku ($r(A) = 1$).

Si bien no está aclarado en el informe como tal, la diferencia entre los tiempos de ejecución de Backtracking con los de Programación Lineal son de una proporción considerable. Mientras que el algoritmo de Backtracking permite realizar análisis hasta aproximadamente $n = 30$, el modelo de programación lineal ya se vuelve inviable a partir de $n = 18$. De hecho, se intentó ejecutar el modelo con un conjunto de datos de dicha magnitud y el proceso no finalizó incluso después de más de 10 horas de ejecución continua.

Además, se evidencia en el cuadro 4 que la aproximación que ofrece el modelo de Programación Lineal alternativo no es muy buena (al menos comparada con el algoritmo del maestro Pakku). Notar que, de todas formas, el mismo respeta el límite planteado al inicio del informe (es una k -aproximación). Para $n = 5$ y $k = 2$ se obtuvo un coeficiente $r(A) \approx 1,99 \approx 2$. Lo que parecía ser una buena idea: minimizar la diferencia entre el subconjunto de máxima suma y el de mínima suma, resultó no serlo en la práctica. Tener en cuenta que, sumado a esto, los modelos de programación lineal entera cuentan con una complejidad exponencial, lo que lo vuelve una aproximación aún peor, sabiendo que el algoritmo greedy analizado posee una complejidad temporal de $O(n \log(n))$.